



K.N. Toosi University of Technology

OFFLINE JAVA GUIDE

Chapter 4

OOP Basics

The four pillars of object-oriented programming, then a deep dive into Java's class system — objects, constructors, methods, static members, access control, encapsulation, and enums.

COURSE

Advanced Programming

SEMESTER

Spring 2026

DEPARTMENT

Computer Engineering

v1.0

Table of Contents

4.1	The Four Pillars of OOP	Concept overview
4.2	Classes & Objects	Blueprints and instances
4.3	Instance Variables & Methods	Fields, behaviors, return types
4.4	Constructors	Default, parameterized, this()
4.5	Method Overloading	Same name, different parameters
4.6	The this Keyword	Referring to the current object
4.7	The static Keyword	Variables, methods, blocks
4.8	Access Modifiers	public, private, protected, default
4.9	Encapsulation	Hiding data, getters/setters
4.10	Enums in Java	Named constants, fields, methods
4.11	Common Mistakes & Tips	Pitfalls every beginner hits

Object-Oriented Programming (OOP) is a way of organizing code around **objects** — bundles of data and the operations that work on that data. Java is built on four core principles, often called the **four pillars**. This chapter covers the first one in depth; the others will appear in later chapters.

Encapsulation

Bundling data and methods inside a class, and restricting direct access to the internal data. The outside world interacts through a controlled interface.

A `BankAccount` hides its `balance`. You can't set it directly — you must call `deposit()` and `withdraw()`.

Inheritance

Creating new classes from existing ones. A child class inherits fields and methods from a parent, and can add or override behavior.

A `Car` class inherits from `Vehicle`, automatically getting `speed` and `fuel` properties.

Polymorphism

"Many forms" — the same operation behaves differently depending on the object. A single interface, multiple implementations behind it.

Calling `draw()` on a `Circle` draws a circle; calling `draw()` on a `Square` draws a square. Same method name, different behavior.

Abstraction

Hiding complex implementation details and exposing only what matters. Focusing on *what* something does, not *how* it does it.

You use `System.out.println()` without knowing how it renders pixels on screen. The complexity is hidden.

NOTE

These pillars are not independent — they work together. A well-designed class is *encapsulated*, uses *abstraction* to hide details, can be *inherited* for extension, and its methods exhibit *polymorphism*. This chapter focuses on encapsulation. Inheritance and polymorphism come in Chapter 5.

What is a Class?

A **class** is a blueprint. It defines what data an object will hold (fields/variables) and what it can do (methods). It is a **template**, not the thing itself.

What is an Object?

An **object** is a concrete instance of a class. If `Student` is the blueprint, then `new Student("Ali", 20)` is one specific student.

```
1  // Defining a class (the blueprint)
2  public class Student {
3      String name;
4      int age;
5  }
6
7  // Creating objects (instances) from the blueprint
8  public class Main {
9      public static void main(String[] args) {
10         Student s1 = new Student();
11         Student s2 = new Student();
12
13         s1.name = "Ali";
14         s1.age = 20;
15
16         s2.name = "Sara";
17         s2.age = 22;
18
19         System.out.println(s1.name + " is " + s1.age);
20         System.out.println(s2.name + " is " + s2.age);
21     }
22 }
```

JAVA

```
Ali is 20 Sara is 22
```

OUTPUT

The new Keyword

`new Student()` does three things:

1. **Allocates memory** on the heap for the object
2. **Initializes fields** to default values (null, 0, false)
3. **Returns a reference** to the new object (stored in the variable)

Multiple References — Same Object vs Different Objects

```
1 Student a = new Student();
2 Student b = new Student(); // different object
3 Student c = a;             // same object as a
4
5 a.name = "Ali";
6 System.out.println(c.name); // "Ali" - c points to the same object
7
8 b.name = "Sara";
9 System.out.println(a.name); // "Ali" - a and b are separate objects
```

MISTAKE

Just like arrays, assigning an object variable to another does **not copy** the object. Both variables point to the **same** object in memory.

Default Values for Fields

FIELD TYPE	DEFAULT VALUE
<code>int</code> , <code>long</code> , <code>short</code> , <code>byte</code>	<code>0</code>
<code>double</code> , <code>float</code>	<code>0.0</code>
<code>boolean</code>	<code>false</code>
<code>char</code>	<code>'\u0000'</code>
Any object (<code>String</code> , arrays, etc.)	<code>null</code>

NOTE

These defaults apply to **instance fields** only. Local variables inside methods do **not** get default values — you must assign them before use, or the compiler will error.

4.3 Instance Variables & Methods

Instance Variables (Fields)

Variables declared directly inside the class (outside any method) are called **instance variables**. Each object gets its own copy.

```
1  public class Rectangle {
2      double width;    // instance variable
3      double height;   // instance variable
4
5      // Instance method – operates on the object's data
6      double area() {
7          return width * height;
8      }
9
10     // Instance method with a parameter
11     void scale(double factor) {
12         width = width * factor;
13         height = height * factor;
14     }
15
16     // Instance method that returns a String
17     String describe() {
18         return String.format("Rectangle %.1f x %.1f (area: %.1f)",
19                               width, height, area());
20     }
21 }
```

JAVA

```
1  Rectangle r = new Rectangle();
2  r.width = 5.0;
3  r.height = 3.0;
4
5  System.out.println(r.area());    // 15.0
6  r.scale(2.0);
7  System.out.println(r.describe()); // Rectangle 10.0 x 6.0 (area: 60.0)
```

JAVA

```
15.0 Rectangle 10.0 x 6.0 (area: 60.0)
```

OUTPUT

Return Types

DECLARATION	MEANING
<code>void doSomething()</code>	Performs an action, returns nothing
<code>int getAge()</code>	Returns an int value
<code>boolean isValid()</code>	Returns a boolean (often named <code>is</code> ... or <code>has</code> ...)
<code>String getName()</code>	Returns a String (object type)
<code>Rectangle createCopy()</code>	Returns an object of the same class

The void Return Type

Methods declared with `void` do not return a value. They execute for their side effects (printing, modifying state, etc.):

```
1 void greet(String name) {
2     System.out.println("Hello, " + name + "!");
3     // No return statement needed (or you can write "return;")
4 }
```

JAVA

MISTAKE

If a method declares a return type other than `void`, you **must** return a value of that type on **every** code path. The compiler checks this:

```
int getAge() { } // compile error: missing return statement
```

Instance vs Local Variables

FEATURE	INSTANCE VARIABLE	LOCAL VARIABLE
Declared where?	Inside class, outside methods	Inside a method or block
Default value?	Yes (0, null, false)	No — must be assigned before use
Scope	Entire class	Only within its block <code>{ }</code>
Access modifier?	Can have <code>public</code> , <code>private</code> , etc.	Cannot have access modifiers
Per-object?	Yes — each object has its own copy	No — created fresh each call

A **constructor** is a special method called automatically when an object is created with `new`. Its job is to **initialize** the object's fields.

Rules

- › Same name as the class — **exactly**
- › **No return type** — not even `void`
- › Can have parameters (like methods)
- › Can be overloaded (multiple constructors with different parameters)

Default Constructor (No-Arg)

```
1 public class Student {
2     String name;
3     int age;
4
5     // Default constructor
6     public Student() {
7         name = "Unknown";
8         age = 0;
9     }
10 }
```

JAVA

NOTE

If you write **no constructor at all**, Java provides an invisible default constructor that sets all fields to their default values. But the moment you write **any** constructor, the invisible one disappears.

Parameterized Constructor

```
1 public class Student {
2     String name;
3     int age;
4
5     // Parameterized constructor
6     public Student(String name, int age) {
7         this.name = name;
8         this.age = age;
9     }
10 }
11
12 // Usage
13 Student s = new Student("Ali", 20);
14 System.out.println(s.name); // "Ali"
```

JAVA

Constructor Overloading

A class can have multiple constructors with different parameter lists:

```
1  public class Student {
2      String name;
3      int age;
4      double gpa;
5
6      public Student() {
7          this.name = "Unknown";
8          this.age = 0;
9          this.gpa = 0.0;
10     }
11
12     public Student(String name) {
13         this.name = name;
14         this.age = 0;
15         this.gpa = 0.0;
16     }
17
18     public Student(String name, int age) {
19         this.name = name;
20         this.age = age;
21         this.gpa = 0.0;
22     }
23
24     public Student(String name, int age, double gpa) {
25         this.name = name;
26         this.age = age;
27         this.gpa = gpa;
28     }
29 }
30
31 Student s1 = new Student();           // Unknown, 0, 0.0
32 Student s2 = new Student("Ali");      // Ali, 0, 0.0
33 Student s3 = new Student("Sara", 22); // Sara, 22, 0.0
34 Student s4 = new Student("Reza", 21, 3.8); // Reza, 21, 3.8
```

Calling One Constructor from Another: `this()`

Use `this(args)` on the first line to call another constructor in the same class — avoids duplicating initialization code:

```
1  public class Student {
2      String name;
3      int age;
4      double gpa;
5
6      public Student(String name, int age, double gpa) {
7          this.name = name;
8          this.age = age;
9          this.gpa = gpa;
10     }
11
12     // Delegates to the 3-param constructor with a default GPA
13     public Student(String name, int age) {
14         this(name, age, 0.0);
15     }
16
17     // Delegates to the 2-param constructor with a default name
18     public Student(int age) {
19         this("Unknown", age);
20     }
21 }
```

MISTAKE

`this(...)` must be the **very first statement** in the constructor. You cannot put any code before it.

```
public Student(String name) { System.out.println("hi"); this(name, 0); } // ERROR
```

Method overloading means defining multiple methods with the **same name** but **different parameter lists**. The compiler decides which one to call based on the arguments you pass.

What Can Differ?

- ✓ Number of parameters
- ✓ Types of parameters
- ✓ Order of parameter types
- ✗ Return type alone — NOT sufficient
- ✗ Parameter names alone — NOT sufficient

```
1 public class Calculator {
2
3     int add(int a, int b) {
4         return a + b;
5     }
6
7     double add(double a, double b) {
8         return a + b;
9     }
10
11     int add(int a, int b, int c) {
12         return a + b + c;
13     }
14
15     // INVALID - only return type differs
16     // long add(int a, int b) { ... } // compile error
17 }
```

JAVA

```
1 Calculator calc = new Calculator();
2 System.out.println(calc.add(2, 3));           // 5           (int, int)
3 System.out.println(calc.add(2.5, 3.5));       // 6.0         (double, double)
4 System.out.println(calc.add(1, 2, 3));        // 6           (int, int, int)
```

JAVA

5 6.0 6

OUTPUT

You've already seen overloading in action: `System.out.println()` is overloaded — it has versions for `int`, `double`, `String`, `boolean`, etc. The compiler picks the right one based on what you pass. Constructors can also be overloaded (as shown in 4.4).

TIP

`this` is a reference to the **current object** — the object whose method or constructor is being called. It has three main uses:

1. Disambiguate Field from Parameter

```
1  public class Student {
2      String name;
3      int age;
4
5      // Without this: parameters shadow the fields
6      // name = name; - assigns the parameter to ITSELF (does nothing)
7      public Student(String name, int age) {
8          this.name = name; // this.name = field, name = parameter
9          this.age = age;
10     }
11 }
```

JAVA

2. Pass the Current Object to Another Method

```
1  public class Printer {
2      void print(Student s) {
3          System.out.println("Printing: " + s.name);
4      }
5  }
6
7  public class Student {
8      String name;
9
10     void sendToPrinter(Printer p) {
11         p.print(this); // passes this Student object
12     }
13 }
```

JAVA

3. Call Another Constructor: this(args)

```
1  public Student(String name, int age) {
2      this(name, age, 0.0); // calls the 3-param constructor
3  }
```

JAVA

QUICK REFERENCE — THIS USAGE

- › `this.field` — refer to an instance field when a parameter has the same name
- › `this.method()` — call another instance method of the same class (rarely needed)
- › `this(args)` — call another constructor (must be first line)
- › `this` — pass the current object reference to another method

Inside a **static** method, there is no `this`. Static methods belong to the class, not to any specific object. Trying to use `this` in a static context is a compile error.

The `static` keyword means "belongs to the class, not to any instance." Static members are shared across all objects of that class.

Static Variables

A static variable has **one copy** shared by all objects. Instance variables have one copy **per object**.

```
1 public class Student {
2     String name;           // instance - each student has their own
3     int id;
4     static int totalStudents; // static - shared by ALL students
5
6     public Student(String name) {
7         this.name = name;
8         totalStudents++;    // every new student increments the shared counter
9     }
10 }
```

JAVA

```
1 Student s1 = new Student("Ali");
2 Student s2 = new Student("Sara");
3 Student s3 = new Student("Reza");
4
5 System.out.println(Student.totalStudents); // 3 - accessed via class name
6 System.out.println(s1.totalStudents);      // 3 - also works but discouraged
```

JAVA

3 3

OUTPUT

Static Methods

A static method belongs to the class. It can **not** access instance variables or call instance methods directly (because there's no `this`):

```
1 public class MathUtils {
2     // Static method - no object needed
3     public static int max(int a, int b) {
4         return (a > b) ? a : b;
5     }
6
7     public static int factorial(int n) {
8         if (n ≤ 1) return 1;
9         return n * factorial(n - 1);
10    }
11 }
12
13 // Call directly via the class name - no new MathUtils() needed
14 System.out.println(MathUtils.max(5, 10));    // 10
15 System.out.println(MathUtils.factorial(5));  // 120
```

JAVA

What Static Methods Cannot Do

```
1  public class Example {
2      int instanceField = 0;
3      static int staticField = 0;
4
5      public static void staticMethod() {
6          System.out.println(staticField); // OK - static can access static
7          // System.out.println(instanceField); // COMPILE ERROR
8          // instanceMethod(); // COMPILE ERROR
9          // System.out.println(this); // COMPILE ERROR
10     }
11
12     public void instanceMethod() {
13         System.out.println(staticField); // OK - instance can access static
14         System.out.println(instanceField); // OK - instance can access instance
15     }
16 }
```

JAVA

Static Initialization Block

Runs once when the class is first loaded (before any object is created):

```
1  public class Config {
2      static String version;
3      static int maxConnections;
4
5      static {
6          version = "1.0.0";
7          maxConnections = 100;
8          System.out.println("Config loaded");
9      }
10 }
11 // "Config loaded" prints the first time this class is referenced
```

JAVA

Static vs Instance — Summary

FEATURE	STATIC	INSTANCE
Belongs to	The class	The object
How many copies?	One (shared)	One per object
Access	<code>ClassName.member</code>	<code>objectRef.member</code>
Can access instance members?	No	Yes
Has <code>this</code> ?	No	Yes
When loaded?	When class is first used	When <code>new</code> is called
Use for	Utilities, constants, counters	Object-specific data & behavior

TIP

Access static members via the **class name** (`Student.totalStudents`), not through an object reference (`s1.totalStudents`). Both work, but the class-name style makes it clear the member is static and avoids confusion.

Access modifiers control **who can see and use** a class, method, or variable. Java has four levels:

MODIFIER	SAME CLASS	SAME PACKAGE	SUBCLASS (OTHER PKG)	WORLD
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

When to Use Which

MODIFIER	TYPICAL USE
private	Instance fields, internal helper methods. The core of encapsulation.
default	Classes and methods used only within the same package.
protected	Fields/methods that subclasses need to access. (Becomes relevant with inheritance.)
public	The class itself, constructors, and methods that form the object's public API.

Practical Rule of Thumb

JAVA

```
1 public class Student {
2     private String name;        // private - hide the data
3     private int age;            // private - hide the data
4
5     public Student(String name, int age) { // public - need to create objects
6         this.name = name;
7         this.age = age;
8     }
9
10    public String getName() { // public - part of the API
11        return name;
12    }
13
14    private boolean isValidAge(int age) { // private - internal helper
15        return age ≥ 0 && age ≤ 150;
16    }
17 }
```


Encapsulation is the practice of hiding an object's internal data and exposing it only through controlled methods. In Java, this means:

1. Make fields `private`
2. Provide `public` **getter and setter methods** to read and write them
3. Add **validation logic** inside the setters

Without Encapsulation (Bad)

```
1  public class Student {
2      public String name;
3      public int age;
4  }
5
6  // Anyone can set invalid data - no protection
7  Student s = new Student();
8  s.name = "";           // empty name - should not be allowed
9  s.age = -5;            // negative age - makes no sense
10 s.age = 500;           // impossible age - no check
```

JAVA

With Encapsulation (Good)

```
1  public class Student {
2      private String name;
3      private int age;
4
5      public Student(String name, int age) {
6          setName(name);
7          setAge(age);
8      }
9
10     // Getter
11     public String getName() {
12         return name;
13     }
14
15     // Setter with validation
16     public void setName(String name) {
17         if (name == null || name.trim().isEmpty()) {
18             throw new IllegalArgumentException("Name cannot be empty");
19         }
20         this.name = name.trim();
21     }
22
23     // Getter
24     public int getAge() {
25         return age;
26     }
27
28     // Setter with validation
29     public void setAge(int age) {
30         if (age < 0 || age > 150) {
31             throw new IllegalArgumentException("Age must be between 0 and 150");
32         }
33         this.age = age;
34     }
35
36     // Read-only: no setter for ID (set only in constructor)
37     private final int id;
38     public Student(String name, int age, int id) {
39         setName(name);
40         setAge(age);
41         this.id = id;
42     }
43     public int getId() {
44         return id;
45     }
46 }
```

```
1 Student s = new Student("Ali", 20, 1234);
2 System.out.println(s.getName()); // "Ali"
3 System.out.println(s.getAge());   // 20
4
5 s.setAge(25);
6 s.setAge(-5); // throws IllegalArgumentException: Age must be between 0 and 150
7 s.setName(""); // throws IllegalArgumentException: Name cannot be empty
8
9 s.id = 999; // COMPILER ERROR - private, no setter (read-only)
```

Why Encapsulation Matters

BENEFIT	EXPLANATION
Data protection	Invalid values are rejected at the setter, preventing the object from entering an inconsistent state.
Controlled access	You can make a field read-only (no setter), write-only (no getter), or add conditional logic.
Implementation flexibility	You can change the internal representation (e.g., switch from <code>String</code> to <code>StringBuilder</code>) without breaking code that uses your class.
Debugging	All changes go through one method — add logging or breakpoints there to trace every modification.

Getters and Setters — Naming Convention

FIELD TYPE	GETTER NAME	SETTER NAME
<code>boolean</code>	<code>isXxx()</code> or <code>getXxx()</code>	<code>setXxx()</code>
Any other type	<code>getXxx()</code>	<code>setXxx()</code>

```
1 private boolean active;
2 private double balance;
3
4 public boolean isActive() { return active; } // "is" for booleans
5 public void setActive(boolean active) { this.active = active; }
6
7 public double getBalance() { return balance; } // "get" for everything else
8 public void setBalance(double balance) { this.balance = balance; }
```

Complete Encapsulated Example — BankAccount

```

1 public class BankAccount {
2     private String accountNumber;
3     private double balance;
4     private final String owner;
5
6     public BankAccount(String owner, String accountNumber, double initialBalance) {
7         if (initialBalance < 0) {
8             throw new IllegalArgumentException("Initial balance cannot be negative");
9         }
10        this.owner = owner;
11        this.accountNumber = accountNumber;
12        this.balance = initialBalance;
13    }
14
15    public void deposit(double amount) {
16        if (amount ≤ 0) {
17            throw new IllegalArgumentException("Deposit must be positive");
18        }
19        balance += amount;
20    }
21
22    public void withdraw(double amount) {
23        if (amount ≤ 0) {
24            throw new IllegalArgumentException("Withdrawal must be positive");
25        }
26        if (amount > balance) {
27            throw new IllegalArgumentException("Insufficient funds");
28        }
29        balance -= amount;
30    }
31
32    public double getBalance() { return balance; }
33    public String getOwner() { return owner; }
34    public String getAccountNumber() { return accountNumber; }
35
36    // owner is read-only – no setter (final, set only in constructor)
37    // accountNumber is read-only – no setter
38    // balance can only change through deposit/withdraw – no direct setter
39 }

```

```

1 BankAccount acc = new BankAccount("Ali", "1234-5678", 1000.0);
2 acc.deposit(500);
3 acc.withdraw(200);
4 System.out.println(acc.getBalance()); // 1300.0
5
6 acc.withdraw(2000); // IllegalArgumentException: Insufficient funds
7 acc.deposit(-100); // IllegalArgumentException: Deposit must be positive

```

1300.0

An **enum** (enumeration) is a special class that represents a **fixed set of constants**. Use it when a variable should only take one of a small, known set of values.

Basic Enum

```
1  public enum Day {
2      SATURDAY, SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY
3  }
4
5  public enum Season {
6      SPRING, SUMMER, AUTUMN, WINTER
7  }
8
9  public enum TrafficLight {
10     RED, YELLOW, GREEN
11 }
```

JAVA

Using an Enum

```
1  Day today = Day.MONDAY;
2
3  System.out.println(today);           // MONDAY
4  System.out.println(today.ordinal()); // 2 (index, starting from 0)
5  System.out.println(today.name());    // "MONDAY" (same as toString)
6
7  // Compare
8  if (today == Day.FRIDAY) {
9      System.out.println("Weekend!");
10 }
11
12 // Switch (no need to write Day.MONDAY - enum type is inferred)
13 switch (today) {
14     case SATURDAY, SUNDAY → System.out.println("Weekend");
15     default → System.out.println("Weekday");
16 }
```

JAVA

MONDAY 2 Weekday

OUTPUT

Enum with Fields and Constructors

Enums can have fields, constructors, and methods — they are full classes:

```

1  public enum Priority {
2      LOW(1, "Low priority"),
3      MEDIUM(3, "Medium priority"),
4      HIGH(5, "High priority"),
5      CRITICAL(10, "Critical priority");
6
7      private final int level;
8      private final String description;
9
10     // Enum constructor - always private (or package-private)
11     Priority(int level, String description) {
12         this.level = level;
13         this.description = description;
14     }
15
16     public int getLevel() { return level; }
17     public String getDescription() { return description; }
18 }

```

```

1  Priority p = Priority.HIGH;
2  System.out.println(p);           // HIGH
3  System.out.println(p.getLevel()); // 5
4  System.out.println(p.getDescription()); // High priority
5
6  // Iterate over all values
7  for (Priority priority : Priority.values()) {
8      System.out.println(priority + " → level " + priority.getLevel());
9  }

```

HIGH 5 High priority LOW → level 1 MEDIUM → level 3 HIGH → level 5 CRITICAL → level 10

Enum with Methods

```

1  public enum Direction {
2      NORTH, SOUTH, EAST, WEST;
3
4      public Direction opposite() {
5          return switch (this) {
6              case NORTH → SOUTH;
7              case SOUTH → NORTH;
8              case EAST → WEST;
9              case WEST → EAST;
10         };
11     }
12 }
13
14 Direction d = Direction.NORTH;
15 System.out.println(d.opposite()); // SOUTH

```

Enum vs Constants — When to Use Which

APPROACH	EXAMPLE	PROBLEMS
Integer constants	<code>public static final int RED = 1;</code>	No type safety: <code>int x = 99;</code> is accepted where a color is expected
String constants	<code>public static final String RED = "RED";</code>	Typo-prone: <code>"REED"</code> compiles fine
Enum	<code>enum Color { RED, GREEN, BLUE }</code>	Type-safe, self-documenting, supports fields/methods, usable in switch

TIP

Whenever you find yourself writing `public static final` constants that are related (days, months, statuses, directions, priorities), replace them with an enum. It's type-safe, more readable, and far easier to extend.

Useful Enum Methods

METHOD	RETURNS	DESCRIPTION
<code>values()</code>	<code>T[]</code>	All enum constants as an array
<code>valueOf(String)</code>	<code>T</code>	Convert string name to enum constant
<code>name()</code>	<code>String</code>	Name of the constant (same as <code>toString()</code>)
<code>ordinal()</code>	<code>int</code>	Index of the constant (0-based)
<code>toString()</code>	<code>String</code>	Name of the constant (can be overridden)

WARNING

Avoid relying on `ordinal()` for logic — it's fragile. If someone reorders the constants, your code breaks silently. Instead, assign explicit fields (like the `level` in the Priority example) and use those.

1. Forgetting new When Creating Objects

```
1 // WRONG - Student is treated as a variable type, not an object
2 Student s;
3 s.name = "Ali"; // NullPointerException - s is null
4
5 // CORRECT
6 Student s = new Student();
7 s.name = "Ali";
```

JAVA

2. Shadowing: Parameter Hides the Field

```
1 public class Student {
2     String name;
3
4     public Student(String name) {
5         name = name; // WRONG - assigns parameter to itself, field stays null
6     }
7
8     // CORRECT
9     public Student(String name) {
10         this.name = name; // this.name = field, name = parameter
11     }
12 }
```

JAVA

3. Calling Instance Method from static Context

```
1 public class Example {
2     void greet() {
3         System.out.println("Hello");
4     }
5
6     public static void main(String[] args) {
7         greet(); // COMPILE ERROR - cannot call instance method without an object
8
9         // CORRECT
10        Example e = new Example();
11        e.greet();
12    }
13 }
```

JAVA

4. Confusing Default Access with public

```
1  class MyClass {           // default access – visible only in its package
2      int x;                 // default access
3      void doSomething() { } // default access
4  }
5
6  // This is NOT public! Only classes in the same package can see MyClass.
7  // Many beginners write "class" and wonder why other files can't find it.
```

5. Trying to Use this in a Static Method

```
1  public class Example {
2      int x;
3
4      public static void process() {
5          this.x = 5; // COMPILER ERROR – no 'this' in static context
6      }
7  }
```

6. Return Type Mismatch

```
1  public int getAge() {
2      return "twenty"; // COMPILER ERROR – String is not int
3  }
4
5  public void doStuff() {
6      return 42;       // COMPILER ERROR – void cannot return a value
7  }
```

7. Enum Constructor Must Be Private

```
1  public enum Color {
2      RED("#FF0000"),
3      GREEN("#00FF00");
4
5      private String hex;
6
7      public Color(String hex) { // COMPILER ERROR
8          this.hex = hex;
9      }
10
11     // Enum constructors are implicitly private.
12     // You CAN write 'private' explicitly, but not 'public'.
13 }
```

8. Not Closing Over Object Aliasing

```
1 public void swap(Student a, Student b) {  
2     Student temp = a;  
3     a = b;  
4     b = temp;  
5     // This swaps the LOCAL references, not the original objects!  
6     // The caller's variables are unaffected.  
7 }  
8  
9 // Java is pass-by-value. For object references, the value is the reference itself.  
10 // Swapping parameters inside a method does not affect the caller.
```

QUICK REFERENCE — OOP DEBUGGING CHECKLIST

- › **NullPointerException** → you forgot `new`, or accessed a field on a `null` reference
- › **Field stays null/zero** → parameter shadowing — use `this.field = param`
- › **"Cannot find symbol" for method** → calling instance method from static context, or wrong object type
- › **Class not found in other files** → class is *default* access, not `public`
- › **"non-static method cannot be referenced from static context"** → create an object first
- › **Changes in one object affect another** → two variables point to the same object (aliasing)
- › **Invalid data in fields** → missing encapsulation — add setters with validation

This document was compiled from the following resources. All content was adapted and restructured for educational use.

[1] **Programiz — Java OOP**

Classes, objects, constructors, encapsulation, method overloading, enums

<https://www.programiz.com/java-programming/oop>

[2] **GeeksforGeeks — OOP Concepts in Java**

Four pillars, access modifiers, static keyword, this keyword, encapsulation examples

<https://www.geeksforgeeks.org/oops-in-java/>

[3] **Oracle — Java Tutorials — Classes and Objects**

Official tutorial on class design, fields, methods, constructors

<https://docs.oracle.com/javase/tutorial/java/javaOO/index.html>

[4] **Baeldung — Java OOP**

Guides on encapsulation best practices, enum usage patterns, access modifiers

<https://www.baeldung.com/java-oop>

[5] **W3Schools — Java OOP**

Quick reference on classes, objects, modifiers, constructors

https://www.w3schools.com/java/java_oop.asp

CREATED BY

Sepehr Ghardashi

sepehr.ghardashi@gmail.com